

Servo-SG90 수정 내역 문서

최초 작성: 2026-04-06 19:31:41 KST

최종 수정: 2026-04-25 KST

1) 문서 목적

이 문서는 STM32 마이크로컨트롤러의 다음 기능을 학습하는 것을 목적으로 합니다.

- **다중 채널 ADC**: Scan mode를 이용하여 복수의 ADC 채널을 순차적으로 변환하는 방법을 학습합니다.
- **TIM 모듈을 이용한 ADC 프리트리거링(Pre-triggering)**: TIM3의 TRGO(Update event)를 ADC 외부 트리거로 연결하여 일정 주기로 ADC 변환을 자동 기동하는 방법을 학습합니다.
- **PWM 출력**: TIM4를 이용한 서보 모터(SG90) 제어용 50Hz PWM 신호 생성 및 듀티 사이클 제어 방법을 학습합니다.

아울러 본 문서는 위 기능 구현 과정에서 반영된 변경 사항과 설계 내용을 정리합니다.

2) 적용 파일

- Core/Src/main.c
- Servo-SG90.ioc
- cmake/gcc-arm-none-eabi.cmake

3) 핵심 변경 사항 요약

- TIM3를 100Hz로 설정하고 ADC1 외부 트리거(TRGO Update)로 연결.
- ADC1 3채널 변환 완료 데이터를 프레임 단위 큐에 저장.
- CMSIS-DSP FIR 예제로 ch0, ch1, ch2에 각각 20Hz 저역통과 필터를 적용.
- TIM3 TRGO 설정 오류를 수정하여 ADC 외부 트리거가 실제로 발생하도록 보정.
- newlib-nano 환경에서 printf의 float 출력을 사용할 수 있도록 링크 옵션을 추가.
- 큐 관련 함수(push, pop, full, empty, count, overflow)를 구현.
- 큐가 가득 찬 경우 가장 오래된 프레임을 폐기하고 최신 프레임을 유지하는 overflow 정책 적용.
- 메인 루프의 1초 주기 토글/상태 출력 코드를 임시 주석 처리.
- 큐에 데이터가 있을 때 ch0,ch1,ch2,fir_ch0,fir_ch1,fir_ch2 형태 CSV 출력으로 변경.
- TIM4 PWM을 서보 표준(50Hz, 1.0ms~2.0ms)으로 유지하고 UART 입력 0~100 비율 제어 유지.
- 함수 구현은 USER CODE 4, 프로토타입은 USER CODE PFP에 배치.
- TIM3/TIM4 설정값을 .ioc 파일에도 동기화.

4) ADC 샘플링 및 큐 구조

4.1 ADC 기본 설정

- ADC1, 12-bit, Scan mode enabled
- External trigger: TIM3 TRGO (rising edge)
- Number of conversion: 3
- EOC selection: single conversion
- Channel sequence
 - Rank 1: ADC_CHANNEL_0

- Rank 2: ADC_CHANNEL_1
- Rank 3: ADC_CHANNEL_4

4.2 프레임 정의

- 1개 프레임은 3개 채널 샘플 묶음
- 타입: AdcFrame_t
 - channel[0] = ADC_CHANNEL_0
 - channel[1] = ADC_CHANNEL_1
 - channel[2] = ADC_CHANNEL_4

4.3 큐 정의

- 큐 길이: 256 프레임
- 타입: AdcFrameQueue_t
- 내부 상태 값
 - head: pop 위치
 - tail: push 위치
 - count: 현재 저장된 프레임 수
 - overflow_count: overflow 발생 횟수

4.4 큐 API

- AdcQueue_Init: 큐 상태 초기화
- AdcQueue_Push: 프레임 삽입
 - Full이면 head를 한 칸 이동하여 가장 오래된 프레임을 제거 후 삽입
 - overflow_count 증가
- AdcQueue_Pop: 프레임 추출
 - Empty면 실패(0) 반환
- AdcQueue_IsFull: full 여부
- AdcQueue_IsEmpty: empty 여부
- AdcQueue_Count: 현재 프레임 수
- AdcQueue_OverflowCount: 누적 overflow 수

4.5 ADC 인터럽트 저장 로직

- HAL_ADC_ConvCpltCallback에서 변환값을 rank index에 맞게 g_adc_current_frame에 저장.
- 3채널이 모두 채워지면 1프레임 완성으로 판단하고 큐 push.
- g_adc_total_frames는 누적 완성 프레임 수로 증가.

5) TIM3 -> ADC 트리거 설정

- 목표 주기: 100Hz (10ms)
- 설정값
 - Prescaler = 8399
 - Period = 99

- TRGO = Update event

계산 개요:

- TIM3 클럭 84MHz 가정
- 카운터 클럭 = $84\text{MHz} / (8399 + 1) = 10\text{kHz}$
- 업데이트 주기 = $(99 + 1) / 10\text{kHz} = 10\text{ms}$
- 결과 주파수 = 100Hz

6) TIM4 서보 표준 PWM 설정

6.1 목표

- 서보 표준 신호 50Hz 생성
- 입력값 0~100을 1.0ms~2.0ms로 선형 매핑

6.2 타이머 설정

- Prescaler = 83
- Period = 19999

계산 개요:

- TIM4 클럭 84MHz 가정
- 카운터 클럭 = $84\text{MHz} / (83 + 1) = 1\text{MHz}$ (1 tick = 1us)
- 주기 = $(19999 + 1) \text{ us} = 20000\text{us} = 20\text{ms} = 50\text{Hz}$

6.3 입력 매핑식

- $\text{pulse_us} = 1000 + ((2000 - 1000) * \text{input} / 100)$

예시:

- 입력 0 -> 1000us
- 입력 50 -> 1500us
- 입력 100 -> 2000us

6.4 TIM4 PWM 설정 함수(MX_TIM4_Init) 상세

TIM4 PWM은 MX_TIM4_Init 함수에서 아래 순서로 설정됩니다.

1. 기본 타이머 파라미터 설정
 - `htim4.Instance = TIM4`
 - `Prescaler = 83, Period = 19999`
 - `CounterMode = UP, ClockDivision = DIV1`
2. 타이머 베이스 초기화
 - `HAL_TIM_Base_Init(&htim4)`로 타이머 카운터 동작 기반을 초기화합니다.
3. 클럭 소스 설정
 - `HAL_TIM_ConfigClockSource`에서 내부 클럭(`TIM_CLOCKSOURCE_INTERNAL`)을 사용합니다.
4. PWM 기능 초기화

- HAL_TIM_PWM_Init(&htim4)로 PWM 모드를 활성화합니다.

5. 채널 파라미터 설정(TIM4_CH1)

- OCMODE = TIM_OCMode_PWM1
- Pulse = 0 (부팅 직후 0% 상태에서 시작)
- OCPolarity = HIGH
- OCFastMode = DISABLE
- HAL_TIM_PWM_ConfigChannel(..., TIM_CHANNEL_1)로 채널 1 설정을 반영합니다.

6. GPIO Alternate Function 연결

- HAL_TIM_MspPostInit에서 PB6를 AF2(TIM4_CH1)로 설정하여 실제 PWM 파형이 핀으로 출력됩니다.

핵심 포인트:

- PWM 주파수(50Hz)는 Prescaler/Period 조합으로 결정됩니다.
- PWM 듀티(서보 펄스폭)는 Pulse(= CCR 값)로 결정되며, 런타임에서 __HAL_TIM_SET_COMPARE로 변경됩니다.

코드 예시 (MX_TIM4_Init 발췌):

```
htim4.Instance = TIM4; // TIM4 주변장치 선택
htim4.Init.Prescaler = 83; // 84MHz / (83+1) = 1MHz
htim4.Init.CounterMode = TIM_COUNTERMODE_UP; // 업카운트 모드
htim4.Init.Period = 19999; // 20000tick = 20ms(50Hz)
htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1; // 추가 분주 없음
htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE; // ARR 프리로드 비활성

HAL_TIM_Base_Init(&htim4); // TIM4 베이스 타이머 초기화
HAL_TIM_ConfigClockSource(&htim4, &sClockSourceConfig); // 내부 클럭 소스 적용
HAL_TIM_PWM_Init(&htim4); // TIM4를 PWM 모드로 초기화

sConfigOC.OCMode = TIM_OCMode_PWM1; // PWM1 모드 선택
sConfigOC.Pulse = 0; // 초기 CCR=0 (초기 출력 최소)
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH; // Active High 출력
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE; // Fast mode 비활성
HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_1); // CH1 설정 반영
```

6.5 PWM 값 저장/적용 함수(SetPwmDutyPercent) 상세

SetPwmDutyPercent 함수는 입력된 퍼센트 값을 서보 펄스폭으로 변환하고, 그 값을 타이머 레지스터에 반영한 뒤 현재 상태를 저장합니다.

동작 순서:

1. 입력 범위 보정(클램프)
 - duty_percent가 100을 초과하면 100으로 제한합니다.
2. 펄스폭 계산
 - pulse_range = SERVO_MAX_PULSE_US - SERVO_MIN_PULSE_US (현재 1000us)
 - pulse = SERVO_MIN_PULSE_US + (pulse_range * duty_percent / 100)
 - 결과적으로 0~100% 입력이 1000~2000us로 선형 매핑됩니다.
3. 하드웨어 적용
 - __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, pulse)
 - TIM4 CH1의 CCR 값이 즉시 갱신되어 다음 PWM 주기부터 새 펄스폭이 반영됩니다.

4. 소프트웨어 상태 저장

- g_pwm_duty_percent = duty_percent
- 현재 적용된 퍼센트를 전역 변수로 저장해 상태 출력/디버깅에 활용합니다.

5. 사용자 피드백 출력

- UART로 "Servo set: X% (Yus)" 메시지를 출력하여 입력 반영 여부를 즉시 확인할 수 있습니다.

코드 예시 (SetPwmDutyPercent 전체):

```
static void SetPwmDutyPercent(uint8_t duty_percent)
{
    uint32_t pulse;                // 최종 CCR(us 단위)
    uint32_t pulse_range;          // 최소~최대 펄스폭 범위

    if (duty_percent > 100U)        // 입력 상한 보호
    {
        duty_percent = 100U;        // 100%로 클램프
    }

    pulse_range = SERVO_MAX_PULSE_US - SERVO_MIN_PULSE_US; // 2000-1000=1000us
    pulse = SERVO_MIN_PULSE_US + ((pulse_range * (uint32_t)duty_percent) / 100U); // 선형 매핑
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, pulse);    // TIM4 CH1 CCR 갱신

    g_pwm_duty_percent = duty_percent; // 현재 퍼센트 상태 저장
    printf("Servo set: %u%% (%luus)\n",
           g_pwm_duty_percent,        // 적용된 퍼센트 출력
           (unsigned long)pulse);     // 계산된 펄스폭(us) 출력
}
```

7) UART 입력 처리

- USART2 non-blocking polling으로 1바이트 수신
- 숫자 누적 후 Enter 입력 시 0~100 범위 검사
- 유효 값은 서보 비율로 반영
- 잘못된 입력은 안내 문구 출력

7.1 입력 처리와 저장 함수 연계(ProcessPwmInput -> SetPwmDutyPercent)

ProcessPwmInput 함수는 UART에서 받은 문자열을 숫자로 파싱한 뒤, 유효 범위(0~100)인 경우 SetPwmDutyPercent를 호출합니다.

연계 흐름:

1. UART non-blocking 1바이트 수신
 - HAL_UART_Receive(..., timeout=0)로 메인 루프를 막지 않고 입력을 확인합니다.
2. 숫자 문자열 버퍼링
 - '0'~'9'만 input_buf에 누적합니다.
 - 공백/탭은 무시합니다.
3. Enter 입력 시 파싱
 - '\r' 또는 '\n' 수신 시 atoi로 정수 변환합니다.
4. 범위 검증 후 적용

- 0~100이면 SetPwmDutyPercent 호출
- 범위를 벗어나면 안내 문구 출력 후 버퍼 초기화

의미:

- UART 입력 검증 로직과 PWM 저장/적용 로직이 분리되어 있어 유지보수가 쉽습니다.
- PWM 관련 변경(매핑식, 제한값, 로그 형식)을 SetPwmDutyPercent 함수 내부에서 일관되게 관리할 수 있습니다.

코드 예시 (ProcessPwmInput 발췌):

```
if (HAL_UART_Receive(&huart2, &rx_char, 1U, 0U) != HAL_OK) // non-blocking 1바이트 수신
{
    return; // 새 입력 없음
}

if ((rx_char == '\r') || (rx_char == '\n')) // Enter 입력으로 한 줄 종료
{
    int value; // 파싱된 정수값

    if (input_len == 0U) // 빈 줄 입력 무시
    {
        return;
    }

    input_buf[input_len] = '\0'; // c 문자열 종료
    value = atoi(input_buf); // 숫자 문자열 -> 정수

    if ((value >= 0) && (value <= 100)) // 유효 범위 검사
    {
        SetPwmDutyPercent((uint8_t)value); // PWM 적용 함수 호출
    }
    else
    {
        printf("Enter servo ratio in range 0~100\n"); // 범위 오류 안내
    }

    input_len = 0U; // 버퍼 길이 초기화
    return; // 한 줄 처리 완료
}
```

8) 메인 루프 동작

초기화 후 실행 순서:

1. TIM4 PWM Start
2. 큐 초기화
3. 서보 초기값 0%
4. TIM3 Start
5. ADC interrupt start

런타임 루프:

- ProcessPwmInput 호출로 UART 입력 처리
- IRQ 보호 구간에서 큐 pop 시도
- pop 성공 시 ch0, ch1, ch2에 대해 각각 CMSIS-DSP FIR 저역통과 필터 적용

- CSV 한 줄 출력
 - 출력 형식: ch0,ch1,ch2,fir_ch0,fir_ch1,fir_ch2
 - 출력 예: 1520,1640,1788,1512.36,1638.42,1779.15

참고:

- 이전의 1초 주기 토글/상태 출력 블록은 현재 주석 처리 상태

9) UART 출력 예시

- 시작 시
 - ADC sampling started: 3ch, 100Hz trigger, 256-frame ring buffer
 - Servo standard mode: 50Hz, 1.0ms~2.0ms pulse
 - CMSIS-DSP FIR enabled: ch0,ch1,ch2, 20Hz low-pass, 100Hz sample rate
 - Enter servo ratio 0~100 then press Enter
- 서보 입력 반영 시
 - Servo set: 35% (1350us)
- ADC 데이터 스트리밍 시
 - 1023,2047,3010,1004.21,2010.54,2962.88
 - 1019,2050,3005,1015.87,2032.14,2991.43

10) 코드 파일 구조

USER CODE 영역 배치:

위치	내용
USER CODE PFP	프로토타입 선언 (ADC callback, queue 함수, PWM/UART 함수)
USER CODE 0	__io_putchar 구현
USER CODE 2	시작 초기화(PWM/TIM/ADC/큐) 및 안내 출력
USER CODE 3	메인 루프(UART 처리 + 큐 pop CSV 출력)
USER CODE 4	큐 함수/ADC callback/PWM/UART 처리 함수 구현

11) CubeMX(.ioc) 동기화

기본값이던 타이머 값을 .ioc에 반영:

항목	반영 값
TIM3.Prescaler	8399
TIM3.Period	99
TIM3.MasterOutputTrigger	TIM_TRGO_UPDATE
TIM4.Prescaler	83
TIM4.Period	19999

12) 2026-04-25 추가 수정 상세

12.1 ADC 인터럽트 미동작 원인 및 수정

증상:

- 시리얼 모니터에 ADC CSV 데이터가 출력되지 않음
- HAL_ADC_ConvCpltCallback가 호출되지 않는 것으로 보이는 상태

원인 분석:

- ADC1은 외부 트리거로 TIM3 TRGO rising edge를 사용하도록 설정되어 있었음
- 하지만 TIM3의 MasterOutputTrigger 설정값이 실제 코드에서 TIM_TRGO_RESET으로 남아 있었음
- 이 경우 TIM3 update event가 ADC 시작 트리거로 전달되지 않아 ADC 정규 변환이 시작되지 않음

수정 내용:

- MX_TIM3_Init 내부의 sMasterConfig.MasterOutputTrigger 값을 TIM_TRGO_UPDATE로 수정
- 그 결과 TIM3 update event가 100Hz로 발생하면서 ADC 외부 트리거가 정상 동작하도록 보정

의미:

- ADC 인터럽트 동작 여부는 ADC 초기화 자체보다도 트리거 소스가 실제로 살아 있는지가 더 중요함
- CubeMX 설정값과 생성 코드가 불일치할 수 있으므로, 트리거 기반 ADC 디버깅 시 TIM Master Mode를 반드시 함께 확인해야 함

12.2 CMSIS-DSP FIR 예제 확장

초기 구현:

- 처음에는 ch0에 대해서만 20Hz 저역통과 FIR 필터를 적용하도록 구성

추가 확장:

- 동일한 FIR 계수를 사용하되, ch0, ch1, ch2 각각에 독립적인 arm_fir_instance_f32 상태를 할당
- 각 채널은 별도의 state buffer를 사용하므로 채널 간 필터 메모리가 섞이지 않음
- 메인 루프에서 큐 pop 성공 시 3개 채널 모두에 대해 1샘플씩 FIR 연산 수행

설정값:

- 샘플링 주파수: 100Hz
- 컷오프 주파수: 20Hz
- FIR tap 수: 11
- 적용 함수: arm_fir_init_f32, arm_fir_f32

출력 형식 변경:

- 기존: ch0,ch1,ch2
- 변경: ch0,ch1,ch2,fir_ch0,fir_ch1,fir_ch2

의미:

- 원본 ADC 값과 필터 출력값을 동시에 비교할 수 있어 필터의 평활화 효과를 즉시 관찰 가능
- 채널별 독립 상태 버퍼를 둔 구조는 향후 각 채널마다 다른 필터를 적용하는 확장에도 유리함

12.3 printf float 출력 문제 및 수정

증상:

- 시리얼 모니터에서 1021,1449,1382,,, 처럼 정수 값 뒤의 실수 출력 필드가 비어 보임

원인 분석:

- 프로젝트는 --specs=nano.specs를 사용하고 있었음
- newlib-nano는 기본 설정만으로는 printf의 %f 포맷 출력을 링크하지 않음
- 따라서 %.2f 형식 문자열을 사용하더라도 float 변환 루틴이 포함되지 않아 빈 필드처럼 보이는 현상이 발생

수정 내용:

- cmake/gcc-arm-none-eabi.cmake에 -u _printf_float 링크 옵션 추가

결과:

- %.2f 형식의 FIR 출력이 정상적으로 시리얼 모니터에 표시됨
- 링크 후 FLASH 사용량이 증가한 것은 float printf 지원 코드가 실제로 포함되었음을 의미함

주의:

- float printf 지원은 코드 크기를 증가시키므로, 최종 제품 단계에서는 필요성에 따라 유지 여부를 판단해야 함
- 디버그/학습 단계에서는 가독성 향상 효과가 크므로 유용함

12.4 오늘 수정 후 기대 동작

- 부팅 직후 시작 안내 문구가 UART에 출력됨
- TIM3 update event가 100Hz로 ADC를 주기적으로 기동함
- ADC 3채널 프레임이 큐에 저장됨
- 메인 루프가 프레임을 pop하여 3채널 FIR 결과를 함께 CSV로 출력함
- 시리얼 모니터 예시:
 - 1021,1449,1382,1018.42,1445.31,1379.88

13) 빌드 검증

- 명령: cmake --build build/Debug
- 결과: 빌드 성공

14) 주의 사항

- TIM 클럭 계산은 현재 클럭 트리 기준이며, 시스템 클럭 변경 시 타이머 값 재검토 필요
- CSV 출력은 샘플 발생 속도에 비례해 UART 트래픽이 증가함
- 큐 overflow는 최신 데이터 유지 정책으로 처리함(오래된 데이터 폐기)
- 필요 시 큐 소비 속도 최적화 또는 DMA 기반 전송 검토 가능
- printf float 지원은 코드 크기를 증가시키므로, 릴리즈 최적화 시 유지 여부를 검토할 필요가 있음

15) 핀맵

기능	핀	CubeMX 라벨명	주변장치/채널	용도
ADC 입력 1	PA0	-	ADC1_IN0 (Rank 1)	아날로그 샘플링 채널 1
ADC 입력 2	PA1	-	ADC1_IN1 (Rank 2)	아날로그 샘플링 채널 2
ADC 입력 3	PA4	-	ADC1_IN4 (Rank 3)	아날로그 샘플링 채널 3
서보 PWM 출력	PB6	-	TIM4_CH1 (AF2)	서보 제어 신호 출력 (50Hz, 1~2ms)
UART TX	PA2	USART_TX	USART2_TX	상태 메시지/로그/CSV 출력
UART RX	PA3	USART_RX	USART2_RX	서보 비율(0~100) 입력
사용자 LED	PA5	LD2	GPIO Output	상태 표시(현재 토글 코드 주석 처리)
사용자 버튼	PC13	B1	GPIO Input/EXTI	기본 사용자 입력(현재 미사용)

내부 연결(핀 없음):

- TIM3 Update Event(TRGO) -> ADC1 External Trigger

16) 시사점 (Takeaways)

15.1 다중 채널 ADC

- Scan mode를 활성화하면 단일 트리거로 복수 채널을 순차 변환할 수 있어, 채널마다 별도의 트리거를 설정할 필요가 없다.
- EOC(End of Conversion) 선택을 "single conversion"으로 설정해야 각 채널 변환 완료 시마다 인터럽트가 발생하며, rank 순서에 따라 데이터를 올바르게 누적할 수 있다.
- 변환 채널 수와 rank 순서를 코드 및 .ioc 양쪽에 일치시키는 것이 핵심이다.

15.2 TIM 모듈을 이용한 ADC 프리거링

- TIM의 Master Mode(TRGO = Update event)를 ADC 외부 트리거로 연결하면 CPU 개입 없이 정확한 주기로 ADC를 자동 기동할 수 있다.
- Prescaler와 Period 값은 시스템 클럭 및 APB 버스 분주 비율을 기준으로 계산해야 하며, CubeMX 클럭 트리 변경 시 반드시 재검토해야 한다.
- 하드웨어 트리거 방식은 소프트웨어 루프 방식 대비 타이밍 지터가 적고, CPU 부하도 줄일 수 있다.

15.3 PWM 출력

- TIM의 카운터 클럭을 1MHz(1 tick = 1us)로 설정하면 펄스 폭을 마이크로초 단위로 직관적으로 계산할 수 있다.
- 서보 모터의 표준 제어 신호(50Hz, 1.0ms~2.0ms)는 _HAL_TIM_SET_COMPARE로 CCR 값을 동적으로 변경하여 구현한다.
- 입력 비율(0~100%)을 펄스 폭(1000us~2000us)으로 선형 매핑하면 직관적인 서보 제어 인터페이스를 구성할 수 있다.

15.4 설계 종합 관점

- ADC 트리거(TIM3)와 PWM 출력(TIM4)을 독립적인 타이머로 분리하면 두 기능의 주기 설정이 서로 간섭하지 않아 유지보수성이 향상된다.
- 인터럽트에서 데이터를 프레임 단위 링 버퍼(큐)에 저장하고 메인 루프에서 소비하는 패턴은 실시간 데이터 스트리밍의 기본 구조로 재 활용 가능하다.
- 향후 DMA 기반 ADC 전송을 적용하면 인터럽트 오버헤드를 추가로 줄이고 더 높은 샘플링 속도를 지원할 수 있다.